INFINITE TRADING PROTOCOL

Infinite Trading Protocol — Claude Code Opus 5.5 UltraCode Audit

Aero ITP/wstETH auto-compounder on Optimism: audit of AeroFactory, AeroVault and AeroZap (v1), differential review against the Velodrome InfiniteVaultStrategy, and the design, adversarial review and deployment of the Aero v2 replacement.

Date	2026-10-24
Chains	Optimism (10), Base (8453)
Auditor	Claude Code, Opus 5.5 (UltraCode multi-agent workflows)
Method	Lens-specific finders, deduplication, per-finding Foundry proof of concept plus adversarial skeptic, fix verification, live-fork testing
Scope	v1: AeroFactory, AeroVault, AeroZap (src/aero). Reference: InfiniteVaultStrategy (src/velodrome). v2: AeroTwapOracle, AeroFactoryV2, AeroVaultV2, AeroZapV2 (src/aero/v2)
Status	v1 superseded on Optimism and Base (empty, no user funds). v2 deployed on both chains and byte-verified; DAO Safe setup batches pending.

1. Executive summary

> **Same factory as the Velodrome vault?** No. The Velodrome ITP/wstETH compounder `0x2811…563f` is a BeaconProxy over InfiniteVaultStrategy, upgradeable by the InfiniteVaultFactory (DAO Safe admin), on a Velodrome V2 pool with VELO rewards. The Aero v1 vault `0xA15a…220D` was created by an immutable AeroFactory `0x8f37…8787` on the new Aero DEX pool factories with xAERO rewards. They share no factory, pool, gauge, reward token or code; only the DAO Safe as owner.
>
> **Was v1 safe?** No principal-drain was found, and deposit, compound and withdraw worked on a live fork. But v1 had confirmed issues that made it unsuitable for real funds: a 1-wei xAERO donation permanently blocked zap deposits (PoC), keepers could swap the reward basket through any factory pool with spot-only pricing, mandatory keepers could never be revoked, pushed fees could freeze withdrawals in USDC vaults, and the live vault's gauge was fixed at zero forever.
>
> **Outcome.** A v2 was designed to the owner's requirements, reviewed adversarially (one High, two Medium, two Low confirmed with PoCs, all fixed and re-verified), and deployed to Optimism for 0.0000082 ETH. All four deployed contracts are byte-identical to the audited source.

	High	Medium	Low	Info
v1 (superseded)	**0**	**4**	**4**	**1**
v2 review (all fixed)	**1**	**2**	**4**	**2**
v2 open	**0**	**0**	**0**	**2 (accepted)**

Overall risk verdict (v2): no known exploitable vulnerability. Residual risks are economic and operational: the ITP/wstETH pool is very thin (about 0.015 wstETH), compounding needs a keeper that refreshes the oracle before harvesting, and the contracts are not upgradeable. This is an AI-performed review; an independent human audit is recommended before significant TVL.

2. Scope and method

- **Source integrity.** Deployed runtime bytecode was compared with local builds (immutables masked). v1 AeroFactory, AeroVault, AeroZap and the Velodrome implementation `0x3647…2944` matched byte for byte including metadata. The v2 oracle, factory, zap and vault implementation also match.
- **v1 audit.** Six lens-specific finders (share accounting, harvest and keeper powers, zap, factory and access control, Aero integration, differential vs the Velodrome v2.1 review) plus a comparison agent; deduplication to nine findings. A session limit stopped automated PoC verification, so v1 findings are marked either *PoC-confirmed* or *confirmed by code review*.
- **v2 review.** Four finders (price and oracle, accounting, access and liveness, zap and integration), deduplication, then for every finding a Foundry PoC agent and an independent skeptic instructed to refute it. All five findings were confirmed by both. A separate agent then reviewed the fixes.
- **Tests.** 96 Aero unit tests pass (v1 63, v2 33 including 512-run fuzzing); Optimism fork suites pass (v2 3/3, v1 lifecycle 2/2) against the real Aero pools.
- **Tools.** Foundry 1.8 (profile `aero`: solc 0.8.26, Cancun, via-IR, 200 runs), cast read-only calls, local forks of Optimism. No keys were handled by the auditor; the deployment signer key never left the signing server process.

3. Velodrome vs Aero v1 vs Aero v2

Aspect	Velodrome (InfiniteVaultStrategy)	Aero v1 (AeroVault)	Aero v2 (AeroVaultV2)
Deployment	BeaconProxy; beacon owned by upgradeable InfiniteVaultFactory (DAO Safe admin)	`new AeroVault` from immutable AeroFactory	EIP-1167 clone of a fixed implementation from immutable AeroFactoryV2
Upgradeability	Yes, all vaults at once via the beacon	No	No (fixes need a new deploy)
DEX / pool	Velodrome V2 pool `0xdAD7…Bf6` (~1.84M ITP / 3.35 wstETH)	Aero classic volatile pool `0x7681…1d36` (~8.2k ITP / 0.015 wstETH)	
Gauge / rewards	Velodrome gauge (currently killed), VELO	Gauge fixed at 0 forever; xAERO never earned	DAO attaches a validated gauge later; staking enabled by DAO only; wrapped gauge calls; emergency unstake
Harvest	Permissionless, TWAP sanity check, router swaps	Keepers only; keeper-chosen pools and minOut; spot quotes	Keepers only by default (owner may open); fixed DAO-approved routes; every hop checked against a TWAP; size caps
Price oracle	Pool `quote()` TWAP	None	AeroTwapOracle from pool reserve cumulatives (Aero pools lack `quote()`); 30 min to 36 h window; no TWAP means no trade
Share accounting	LP only, harvest on deposit	LP plus a pro-rata basket of uninvested tokens	LP only; harvested LP unlocks linearly over 24 h; deposits and redeems never trade
Fees	Performance (max 50%), withdrawal (max 5%)	DAO 2% first, then strategist ≤10% and keeper of the rest; entry/exit ≤2% split DAO/strategist; pushed	Independent % of gross: DAO 2% (≤10%), strategist 0% (≤10%), keeper 0.5% (≤2%); exit ≤1% kept by holders; no entry fee; all accrued and pull-claimed
Keeper control	Owner-set	Mandatory keepers, never removable	DAO adds/removes factory keepers; owner adds vault extras
Pause	Pausable (withdraw always open)	None	None; withdrawals depend on nothing but the gauge's withdraw
Who can launch	Factory admin	Anyone	Anyone; only the DAO marks vaults official

4. Deployed configuration

Contract	Address	Notes
Optimism (10)		

AeroTwapOracle	`0x351F1F1aBf8cc65AacD13cAA19bB4639F7C0b5bf`	Snapshots written at each vault creation
AeroFactoryV2	`0x337Fe450bCD603B1c637AB0F4C073d926a1aE3Dc`	DAO and treasury = Safe `0xb5dB…CEfB`; DAO fee 2%; default keeper `0xE6C3…A340`
AeroVaultV2 implementation	`0xD24D4D249D526C8Bf8dfb4682D11E3f9c6337E9a`	Not a vault; initializers disabled
AeroZapV2	`0x95E97Def5456d5860a26DD3108A71Dd13b3E3592`	Optional single-token router
ITP/wstETH vault	`0x4bd84dD4fF4FfbcD2ee08dB22Ce3b9a2eeB10a58`	Route xAERO → USDC → ITP
ITP/USDC vault	`0x579f6D1646fDEb59B37a4DC880DbB6857004BafD`	Route xAERO → USDC
ITP/WBTC vault	`0x6E276D140230C6f967ee2c79AEd69C90A24e115D`	Route xAERO → USDC → ITP
Base (8453)		
AeroTwapOracle	`0x4751B11220919d8A76Bf3998d85Ecb6524D0Ed82`	
AeroFactoryV2	`0xB60ce95161506EeC22A2D68A784DD21cf78adcA9`	Same DAO, treasury and keeper
AeroVaultV2 implementation	`0x3f81c3E6cDaB6139E62fec8800F6093b9b2405C0`	Not a vault
AeroZapV2	`0xAdF1288917495F806b6014f354638F905B9Ce3fa`	
AERO/USDC vault	`0xA7e2E4Cb6B4938AAaDE3EbDEDe373205818129fC`	Gauge `0xd424…5cD1` (DAO batch), AERO is token0 so no route; fork-tested with real emissions

Deployment from `0xE6C3…A340`: Optimism 12 transactions, 0.0000095 ETH; Base 6 transactions, 0.0000488 ETH. Every vault starts with DAO 2%, strategist 0%, keeper 0.5%, exit 0%, 24 h unlock, and is pending-owned by the DAO Safe. All implementations are byte-identical to the audited source on both chains. Superseded v1 on Optimism (all empty): vault `0xA15a…220D`, ITP/USDC vault `0xD24a…eEf0`, zap `0xb797…c561`, factory `0x8f37…8787`. Superseded Base v1: AERO/USDC vault `0x8a69…395f` (empty). v1 remains on Ethereum and Robinhood.

5. Findings summary

ID	Title	Severity	Status
ITP-AERO-01	1-wei xAERO donation permanently blocks zap deposits (v1)	Medium	PoC-confirmed; fixed in v2 (no basket)
ITP-AERO-02	Keeper can route the reward basket through any factory pool with spot-only pricing (v1)	Medium	Code review; fixed in v2
ITP-AERO-03	Mandatory keepers can never be revoked (v1)	Medium	Code review; fixed in v2
ITP-AERO-04	Pushed fees to a blocklisted strategist freeze withdrawals in USDC vaults (v1)	Medium	Code review; fixed in v2 (pull fees, test)
ITP-AERO-05	Unwrapped gauge calls; a killed gauge blocks deposits and harvest (v1)	Low	Fixed in v2
ITP-AERO-06	Live vault gauge immutable at zero: never earns xAERO (v1)	Low	Fixed in v2 (DAO setGauge)
ITP-AERO-07	Fee changes take effect instantly (v1)	Low	Acknowledged: owner chose no timelock; v2 caps bound impact
ITP-AERO-08	Permissionless createVault allows look-alike vaults (v1)	Low	Fixed in v2 (DAO official flag)
ITP-AERO-09	Pool ~200x thinner than the Velodrome pool	Info	Open: add liquidity before marketing
ITP-AEROV2-01	Forced-skip of pre-deposit compounding lets anyone capture pending income	High	PoC-confirmed; fixed and re-verified
ITP-AEROV2-02	Reward clamp never converges when an intermediate hop binds	Medium	PoC-confirmed; fixed
ITP-AEROV2-03	12 h TWAP window too short for a daily keeper	Medium	PoC-confirmed; fixed (36 h + keeper flow)
ITP-AEROV2-04	Deposits/redeems trigger swaps, bypassing the keeper gate	Low	PoC-confirmed; fixed
ITP-AEROV2-05	Revoked route pools still used by existing routes	Low	PoC-confirmed; fixed
ITP-AEROV2-06	Profit unlock shorter than the harvest interval	Low	Fixed (24 h default)
ITP-AEROV2-07	Owner can shorten the unlock right after a harvest	Low	Fixed (applies next harvest)
ITP-AEROV2-08	Harvest restarts the unlock schedule	Info	Accepted (Yearn-style behaviour)
ITP-AEROV2-09	Deposit just after a harvest shares the unlocking profit	Info	Accepted trade-off; mitigated by frequent harvests

6. Detailed findings: v1 (AeroVault, AeroZap, AeroFactory)

ITP-AERO-01 · 1-wei xAERO donation blocks zap deposits Medium

Location: `AeroVault.deposit` basket pull (`needed = mulDiv(basket, net, assets, Ceil)`); AeroZap.

On Optimism xAERO is the third basket token. Once the vault holds any xAERO, every deposit must also supply a rounded-up share of it, which the zap cannot. xAERO is freely transferable, and the dead shares mean the dust never fully leaves. Withdrawals keep working.

PoC: live-fork test `test_xAeroDustBlocksZapDeposits` (saved as `test/aero/AeroOptimismLiveCycle.fork.t.sol`): after one 1-wei transfer, the next zapIn reverts; alice still redeems.

Resolution: v2 removes the basket entirely; regression test `test_DustDonationsDoNotBlockDeposits`.

ITP-AERO-02 · Keeper-chosen routes with spot-only pricing Medium

Location: `AeroVault.harvest / _swap`.

`_swap` accepts any pool the Aero factories report as a pool (pool creation is permissionless) and any non-zero minOut; its only price check is that same pool's spot quote. A compromised keeper EOA could route the uninvested basket through a pool it created with skewed reserves. LP principal is not reachable.

Resolution: v2 swaps only through owner-set routes of DAO-approved pools (re-checked at swap time), and each hop must be within the slippage bound of a TWAP, with spot within 3% of the TWAP and size capped at 0.5% of reserves.

ITP-AERO-03 · Mandatory keepers can never be revoked Medium

AeroFactory has `addMandatoryKeeper` but no removal, vaults reject removing a mandatory keeper, and both contracts are immutable. A leaked keeper key would stay trusted in every vault forever.

Resolution: v2 `AeroFactoryV2.setKeeper(k, false)` (DAO), except the current default keeper; tested.

ITP-AERO-04 · Pushed fees can freeze redemptions Medium

Every redeem pushes strategist and exit-fee transfers. With a blocklistable token such as USDC, an owner (or compromised Safe) setting a blocklisted strategist makes every redeem revert. Not exploitable on ITP/wstETH; relevant to the v1 ITP/USDC vault.

Resolution: v2 accrues all fees and pays them only on `claimFees`/`payDao`; test `test_BlockedStrategistCannotFreezeRedeem`.

ITP-AERO-05…09 · Lower-severity items

- **05 Low:** gauge `deposit`/`claimEmissions` unwrapped; a killed gauge blocks deposits and harvests (regression vs Velodrome v2.1). v2 wraps them and adds emergency unstake.
- **06 Low:** the live v1 vault was created with gauge 0, which is immutable, so it can never stake or earn xAERO. v2 lets the DAO attach a validated gauge later.
- **07 Low:** fee changes are instant. The owner explicitly chose no timelock; v2 caps (strategist 10%, keeper 2%, exit 1%) bound the impact.
- **08 Low:** anyone can create a look-alike vault that factory listings treat as legitimate. v2 keeps permissionless launching (owner requirement) and adds a DAO-only `isOfficial` flag for frontends.
- **09 Info:** the Aero pool is about 200x thinner than the Velodrome pool; a 0.002 wstETH zap round trip lost about 16% to price impact on a fork.

7. Detailed findings: v2 adversarial review

ITP-AEROV2-01 · Forced-skip compounding lets anyone capture pending income High Fixed

Original design: deposits and redeems ran a best-effort compound before pricing shares on LP only. Price-sensitive steps skip (not revert) when spot is more than 3% from the TWAP.

Exploit: in one transaction an attacker skews the pool, deposits (compound skipped, pending income excluded from the price), restores the pool and redeems (compound now runs and the attacker receives their pro-rata share of it). Mock PoC: the attacker took about 92% of 2,000 ITP of pending income. Fork PoC against the real Optimism pool: about 99%.

Fix: deposits and redeems no longer compound or trade at all; only keeper harvests do. LP minted by a harvest is added to `lockedProfit` and unlocks linearly over `profitUnlockTime` (24 h), and `totalAssets()` excludes the locked part. Re-running the reviewers' PoCs: the attacker now ends 146 ITP down (mock) and receives exactly the LP deposited (fork).

Regressions: `test_JitSandwichCapturesNothing`, `test_ProfitUnlocksLinearly`, `test_UserActionsNeverSwapOrCompound`.

ITP-AEROV2-02 · Reward clamp does not converge on intermediate hops Medium Fixed

Rescaling the route input by `cap/input` overshoots a later hop's cap because swap output is concave, and the loop did not re-plan after its last rescale, so rewards were never swapped. Fix: four attempts, rescaling with a 2% margin and always re-planning. Test `test_ClampConvergesOnIntermediateHop`.

ITP-AEROV2-03 · TWAP window too short for a daily keeper Medium Fixed

With snapshots only at harvest time and a 12 h maximum window, a daily keeper always found the previous snapshot too old, so nothing ever compounded. Fix: `MAX_WINDOW` 36 h (ring of 240 at 10-minute granularity) plus an operational rule: call `oracle.updateMany([pool, …route])` 30–60 minutes before `harvest`. The dashboard keeper plan implements this order.

ITP-AEROV2-04…09 · Lower-severity items

- **04 Low, fixed:** deposits and redeems could trigger the vault's swaps, letting anyone time them against the slippage bound. Resolved by the 01 fix.
- **05 Low, fixed:** `approvedPool` was only checked when a route was set. Now re-checked per hop (`NOT_APPROVED` skip); test `test_RevokedPoolStopsRoute`.
- **06 Low, fixed:** a 12 h unlock with a daily keeper doubled a just-before-harvest depositor's share. Default is now 24 h.
- **07 Low, fixed:** an untrusted vault owner could shorten the unlock right after a harvest. Changes now take effect at the next harvest.
- **Additional hardening:** staking can only be enabled with a reward route when the reward token is not a pool token, and the route cannot be cleared while staking.
- **08–09 Info, accepted:** each harvest restarts the unlock schedule, and a depositor who joins after a harvest shares the unlocking profit in proportion to capital and time (the standard Yearn trade-off).

Proof-of-concept excerpt (ITP-AEROV2-01 regression)

```
function test_JitSandwichCapturesNothing() public {
    _deposit(alice, 10_000 ether);
    _seedOracle();
    itp.mint(address(pool), 1000 ether); weth.mint(address(pool), 0.1 ether);
    pool.addFees(address(vault), 1000 ether, 0.1 ether);
    uint256 eveLp = _lp(eve, 50_000 ether);
    vm.startPrank(eve); pool.approve(address(vault), eveLp);
    uint256 s = vault.deposit(eveLp, 1, eve); vm.stopPrank();
    vm.prank(keeper); vault.harvest(1);              // harvest lands while Eve is in
    vm.prank(eve); uint256 out = vault.redeem(s, 1, eve);
    assertLe(out, eveLp, "no instant profit");
    vm.warp(block.timestamp + 24 hours);
    assertGt(_lpPerShare(), 1e18, "income went to the holders who stayed");
}
```

8. Go-live checklist

#	Action	Owner	Status
1	Execute the Safe batch: `acceptOwnership`, approve xAERO/USDC and ITP/USDC pools, `setOfficial`, `setRewardRoute`	DAO Safe signers	**Pending**
2	Seed meaningful liquidity in the Aero ITP/wstETH pool before promoting the vault	Treasury / LP	**Pending**
3	Keeper: hourly job that updates the oracle, harvests only when pending income is at least 3x the gas cost (minimum 20 h apart), and pays out fees above thresholds	Keeper	**Built and tested; schedule pending**
4	When Aero creates and activates the pool's gauge: DAO `setGauge`, then `setStaking(true)`	DAO Safe	When available
5	Frontend: show v2 fees (DAO 2%, strategist, keeper, exit, 24 h unlock) and use the official flag; v1 Optimism vaults hidden	Frontend	**Implemented, uncommitted**
6	Replace the v1 ITP/USDC (Optimism) and Base AERO/USDC vaults with v2 instances; add ITP/WBTC	Owner	**Done (v2 deployed)**
7	Independent human audit before significant TVL; update the "unaudited" UI notice accordingly	Owner	Recommended

9. Disclaimer

This report reflects the code and on-chain state reviewed on 2026-10-24. It was produced by an AI system (Claude Code, Opus 5.5) using automated multi-agent analysis, proof-of-concept tests and live-fork simulation. It is not a guarantee that the contracts are free of defects, does not cover economic design beyond what is stated, the Aero DEX contracts themselves, token contracts, front-end or off-chain keeper infrastructure, and does not constitute financial advice. The contracts are not upgradeable; any later issue requires a new deployment and migration. An independent human audit is recommended before significant value is deposited.